GuardianBand Lite — ESP32 SOS Beacon — PRD

Objective

Create firmware for an ESP32 DevKit that acts as a personal SOS beacon: the wearer presses a button to request help, the device raises a local alarm and broadcasts its safety state over BLE advertising so a nearby caregiver phone or laptop can see it without pairing.

Hardware

Item	Connection
ESP32 DevKit (ESP32-WROOM-32)	USB to laptop (power + UART)
Active buzzer (3.3 V, digital on/off)	GPIO25 to buzzer +, buzzer − to GND
Alert LED with 220 Ω resistor	GPIO26 to resistor to LED anode, LED cathode to GND
Push button	On-board BOOT button (GPIO0, active low, use internal pull-up)

Device States

Code	State	Meaning
0	NORMAL	No event. LED off, buzzer off.
3	SOS	The wearer requested help.

Requirements

1. In NORMAL, a long press (≥ 2 s) of the button changes the state to SOS.

2. In SOS, beep the buzzer 500 ms on / 500 ms off and blink the LED at 4 Hz.

3. In SOS, a long press (≥ 3 s) of the button acknowledges the alert and returns to NORMAL.

4. Debounce the button for 50 ms.

5. Each transition into SOS increments an 8-bit event counter (wraps at 255).

6. Initialize BLE after boot and start non-connectable BLE advertising automatically. No GATT services, connections or pairing.

7. Read the ESP32 Bluetooth MAC address at runtime and name the device GuardianBand_XX, where XX is the last 2 uppercase hexadecimal digits of that MAC address.

8. Include the complete local name and Manufacturer Specific Data in the advertising payload. Manufacturer Specific Data layout (8 bytes, little-endian):

Byte	Field	Value
0–1	Company ID	0xFFFF (test ID), sent as FF FF
2	Protocol version	0x01
3	State code	0 NORMAL, 3 SOS
4	Event counter	0–255
5	Last impact peak	always 0
6–7	Uptime	seconds since boot ÷ 10, uint16 little-endian

9. Update the advertising payload within 500 ms of every state change and at least once every 10 seconds otherwise.

10. Keep BLE advertising continuously for the whole time the device is powered.

11. UART at 115200 baud. At boot print "GuardianBand Lite v1.0", the Bluetooth MAC address, the generated BLE name and "BLE ADVERTISING STARTED".

12. On every state change print `STATE <old> -> <new> | events=<n>`.

13. Test hook: when the character `s` is received on UART, behave as a long press in NORMAL (enter SOS).

14. Use ESP-IDF v5.5 with the NimBLE host stack. Keep button, alert output and BLE advertising in separate modules.

Out of Scope

GATT, connections, pairing, Wi-Fi, cloud, displays, sensors, deep sleep, OTA.

Acceptance Criteria

1. The ESP32 boots, prints the banner, MAC and GuardianBand_XX name, and "BLE ADVERTISING STARTED" with no reboot loop.

2. A BLE scanner app shows GuardianBand_XX, and XX matches the last 2 digits of the Bluetooth MAC printed on UART.

3. Long-pressing BOOT for 2 seconds enters SOS: buzzer and LED alarm start and UART prints the state change.

4. Long-pressing BOOT for 3 seconds in SOS returns to NORMAL.

5. Sending `s` over UART enters SOS.

6. The manufacturer data seen by a BLE scanner shows the correct state byte within 1 second of each state change and the event counter increases by one per SOS.